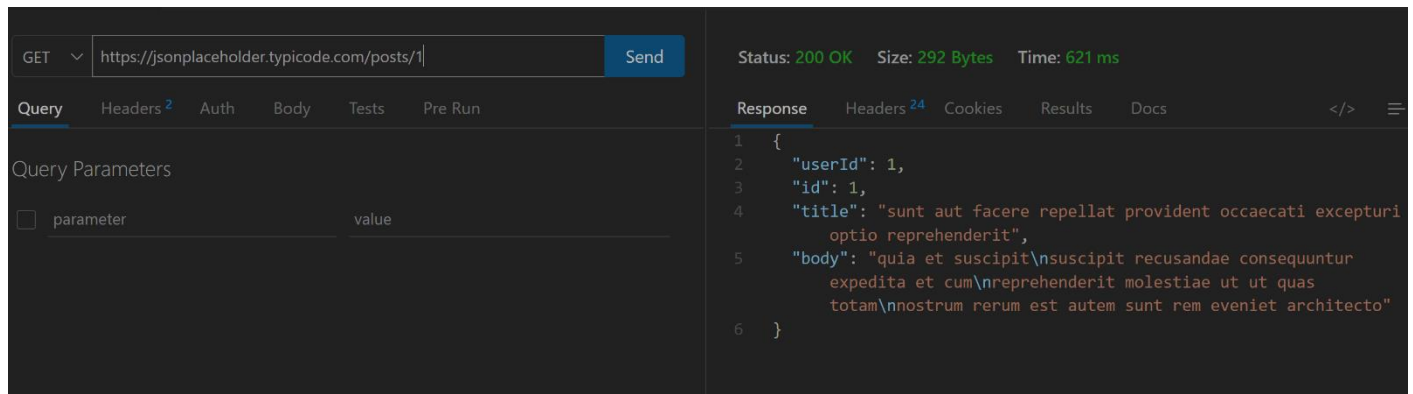


Practice Assignment

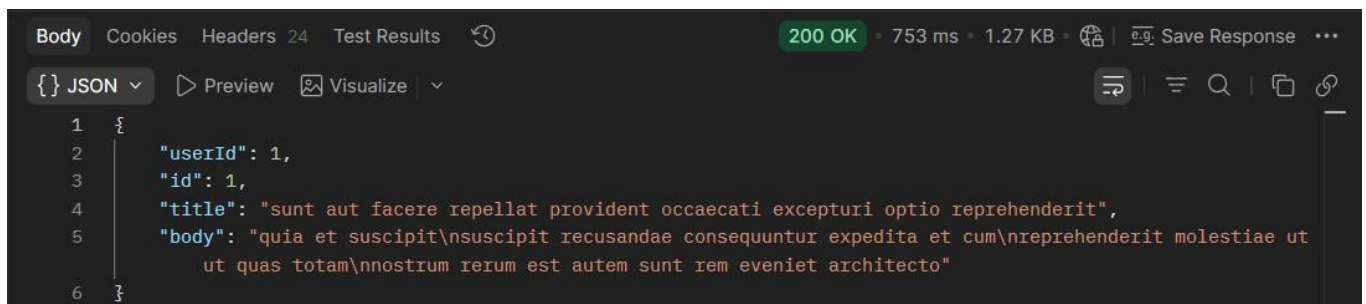
API Fundamentals & Postman Mastery

TASK 1 Reading Data Without Headers

a) Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.



b) Record the status code you get back.



c) In one sentence, explain why this request works even with nothing in the Headers tab.

→ Because GET method is used to retrieve data and it is only method without header so we not need to pass any header it work efficiently without any problem.

TASK 2 Filtering With Query Parameters

a) Send a GET request to /comments, and add query parameter in Params tab: postId = 1.

GET <https://jsonplaceholder.typicode.com/comments?postId=1> Send

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

☒ postId 1

☐ parameter value

Status: 200 OK Size: 1.47 KB Time: 335 ms

Response Headers 25 Cookies Results Docs </> ≡

```
1  [
2  {
3    "postId": 1,
4    "id": 1,
5    "name": "id labore ex et quam laborum",
6    "email": "Eliseo@gardner.biz",
7    "body": "laudantium enim quasi est quidem magnam voluptate
            ipsam eos\ntempora quo necessitatibus\ndolor quam autem
            quasi\nreiciendis et nam sapiente accusantium"
8  },
9  {
10   "postId": 1,
11   "id": 2,
12   "name": "quo vero reiciendis velit similique earum",
13   "email": "Jayne_Kuhic@sydney.com",
14   "body": "est natus enim nihil est dolore omnis voluptatem
            numquam\net omnis occaecati quod ullam at\nvolutatem
            error expedita pariatur\nnihil sint nostrum voluptatem
            reiciendis et"
15  },
16  {
17   "postId": 1
```

b) Confirm the URL automatically updates to include ?postId=1.

My Collection > Task2 Save Share

GET <https://jsonplaceholder.typicode.com/comments?postId=1> Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☒ postId	1			
Key	Value	Description		

c) Now combine two filters at once on /todos: userId = 1 and completed = false.

The screenshot shows a REST client interface with the following details:

- Request:** GET `https://jsonplaceholder.typicode.com/todos?userId=1&completed=false`
- Status:** 200 OK, **Size:** 13.04 KB, **Time:** 1.53 s
- Query Parameters:**

Parameter	Value
userId	1
completed	false
parameter	value
- Response (JSON):**

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  }
]
```

d) In one sentence, explain what changed in the results compared to not using any params.

→ Using query parameters filters the data returned by the server, so only records matching the specified conditions are included instead of all records

TASK 3 Sending Data With Headers & a Body

a) Send a POST request to /posts.

The screenshot shows a REST client interface with a dark theme. At the top, there is a dropdown menu set to 'POST' and a text input field containing the URL 'https://jsonplaceholder.typicode.com/posts'. To the right of the URL is a blue 'Send' button. Below the URL bar, there are several tabs: 'Query' (which is selected and underlined), 'Headers', 'Auth', 'Body', 'Tests', and 'Pre Run'. The 'Query' tab is active, displaying the title 'Query Parameters'. Below this title, there is a table with two columns: 'parameter' and 'value'. The 'parameter' column has a checkbox to its left, and the 'value' column is currently empty.

b) In the Headers tab, add: Content-Type: application/json

The screenshot shows the same REST client interface, but now the 'Headers' tab is selected and underlined. The title 'HTTP Headers' is displayed, and there is a checkbox labeled 'Raw' to its right. Below the title, there is a table with two columns: 'header' and 'value'. The 'header' column has a checkbox to its left. One row is already filled in: the checkbox is checked, the header is 'Content-Type', and the value is 'application/json'. Below this row, there is another row with an unchecked checkbox, the header 'header', and an empty 'value' field.

c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.

The screenshot shows the REST client interface with the 'Body' tab selected and underlined. Below the tab, there are several sub-tabs: 'JSON' (which is selected and underlined), 'XML', 'Text', 'Form', 'Form-encode', 'GraphQL', and 'Binary'. The 'JSON' sub-tab is active, displaying the title 'JSON Content' and a 'Format' button to its right. Below the title, there is a text area containing the following JSON content:

```
1 {
2   "userId" : 11,
3   "title" : "username",
4   "body" : "Ravi Darji"
5 }
```

d) Record the status code and the new id the server assigned.

The screenshot shows a REST client interface with a dark theme. At the top, a POST request is configured to `https://jsonplaceholder.typicode.com/posts`. The 'Body' tab is selected, showing a JSON payload. The 'Send' button is visible. On the right, the response status is `201 Created`, with a size of `78 Bytes` and a time of `2.54 s`. The 'Response' tab is selected, displaying the JSON response. The 'JSON Content' tab is also selected in the left pane, showing the same JSON payload.

POST `https://jsonplaceholder.typicode.com/posts` Send

Status: **201 Created** Size: **78 Bytes** Time: **2.54 s**

Query Headers¹ Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

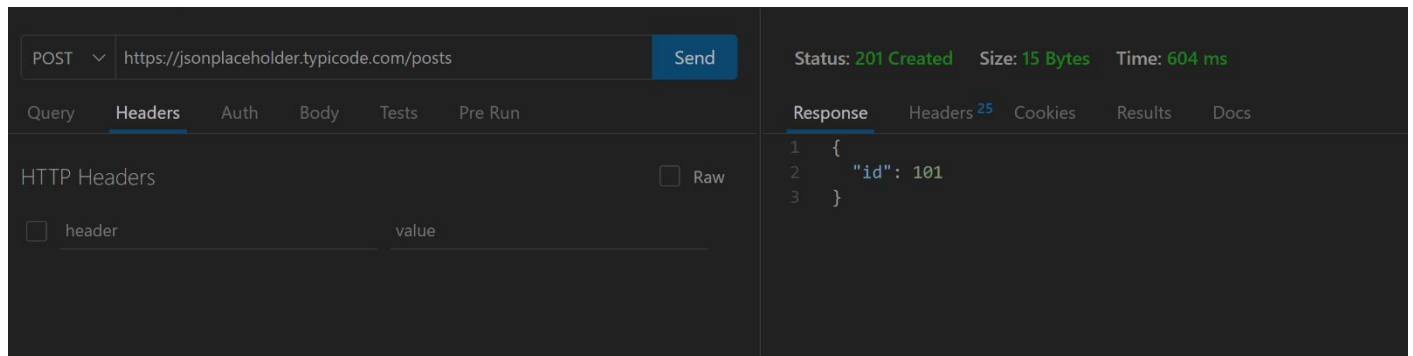
```
1 {
2   "userId" : 11,
3   "title" : "username",
4   "body" : "Ravi Darji"
5 }
```

Response Headers²⁵ Cookies Results Docs

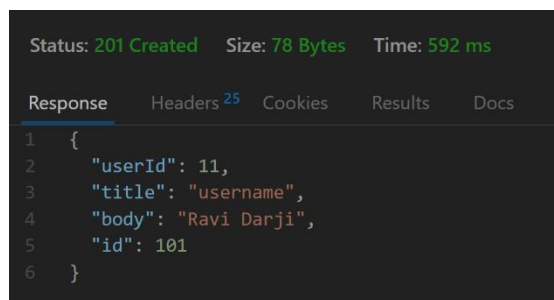
```
1 {
2   "userId": 11,
3   "title": "username",
4   "body": "Ravi Darji",
5   "id": 101
6 }
```

TASK 4 Headers vs. No Headers

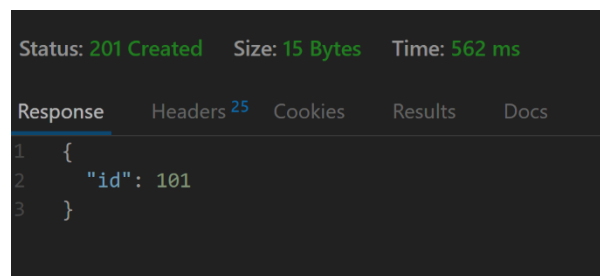
- a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.



- b) Compare the response to Task 3 — is it different? Record what you observe



TASK 3 RESPONSE



TASK 4 RESPONSE

→ Both Give Status Code : **201 Created**

→ The response is the same as Task 3, and the server still returns the created post with id : 101. And this time we not passes Content-Type so it not sawing extra detail which we passed in title, id, body in Task 3.

c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

→ Headers tell the server how to interpret the data being sent in the request. The Content-Type: application/json header indicates that the request body is in JSON format. JSONPlaceholder accepts the request even without this header because it is a practice API, but real-world APIs often reject or incorrectly process requests if the required headers are missing .

TASK 5 PUT vs. PATCH

- a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).

The screenshot shows a REST client interface with a PUT request to `https://jsonplaceholder.typicode.com/posts/1`. The request body is a JSON object: `{ "id": 1, "title": "Updated title", "body": "This is Updated body", "userId": 1 }`. The response status is `200 OK` with a size of `90 Bytes` and a time of `1.29 s`. The response body is a JSON object: `{ "id": 1, "title": "Updated title", "body": "This is Updated body", "userId": 1 }`.

```
PUT https://jsonplaceholder.typicode.com/posts/1
Send

Query Headers Auth Body1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary
JSON Content Format
1 {
2   "id" : 1,
3   "title" : "Updated title",
4   "body" : "This is Updated body",
5   "userId" : 1
6 }
```

```
Status: 200 OK Size: 90 Bytes Time: 1.29 s
Response Headers23 Cookies Results Docs
1 {
2   "id": 1,
3   "title": "Updated title",
4   "body": "This is Updated body",
5   "userId": 1
6 }
```

- b) Send a PATCH request to the same /posts/1 that changes only the title.

The screenshot shows a REST client interface with a PATCH request to `https://jsonplaceholder.typicode.com/posts/1`. The request body is a JSON object: `{ "title": "Updated title" }`. The response status is `200 OK` with a size of `231 Bytes` and a time of `1.30 s`. The response body is a JSON object: `{ "userId": 1, "id": 1, "title": "Updated title", "body": "quia et suscipit\nnsuscipit recusandae consequuntur expedita et cum\nnreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto" }`.

```
PATCH https://jsonplaceholder.typicode.com/posts/1
Send

Query Headers Auth Body1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary
JSON Content Format
1 {
2   "title" : "Updated title"
3 }
```

```
Status: 200 OK Size: 231 Bytes Time: 1.30 s
Response Headers23 Cookies Results Docs
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "Updated title",
5   "body": "quia et suscipit\nnsuscipit recusandae consequuntur
expedita et cum\nnreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

- c) In one sentence, explain the difference between what PUT sent and what PATCH sent.
→ PUT sends the complete resource to replace it, whereas PATCH sends only the fields that need to be updated.

Final Challenge

TASK 6 The Mini Feedback System

Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

- Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.

The screenshot shows a REST client interface. The request is a GET to `https://jsonplaceholder.typicode.com/comments?postId=1`. The response status is 200 OK, with a size of 1.47 KB and a time of 1.02 s. The response body is a JSON array of two comment objects.

```
GET https://jsonplaceholder.typicode.com/comments?postId=1
Send

Query Parameters
[+] postId 1
[-] parameter value

Response
1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate
            ipsam eos\ntempora quo necessitatibus\ndolor quam autem
            quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2,
12    "name": "quo vero reiciendis velit similique earum",
13    "email": "Jayne_Kuhic@sydney.com",
14    "body": "est natus enim nihil est dolore omnis voluptatem
            numquam\net omnis occaecati quod ullam at\nvoluptatem
            error expedita pariatur\nnihil sint nostrum voluptatem
            reiciendis et"
15  },
16  {
17    "postId": 1
```

- Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.

The screenshot shows a REST client interface. The request is a POST to `https://jsonplaceholder.typicode.com/comments`. The request body is a JSON object. The response status is 201 Created, with a size of 126 Bytes and a time of 1.49 s. The response body is a JSON object.

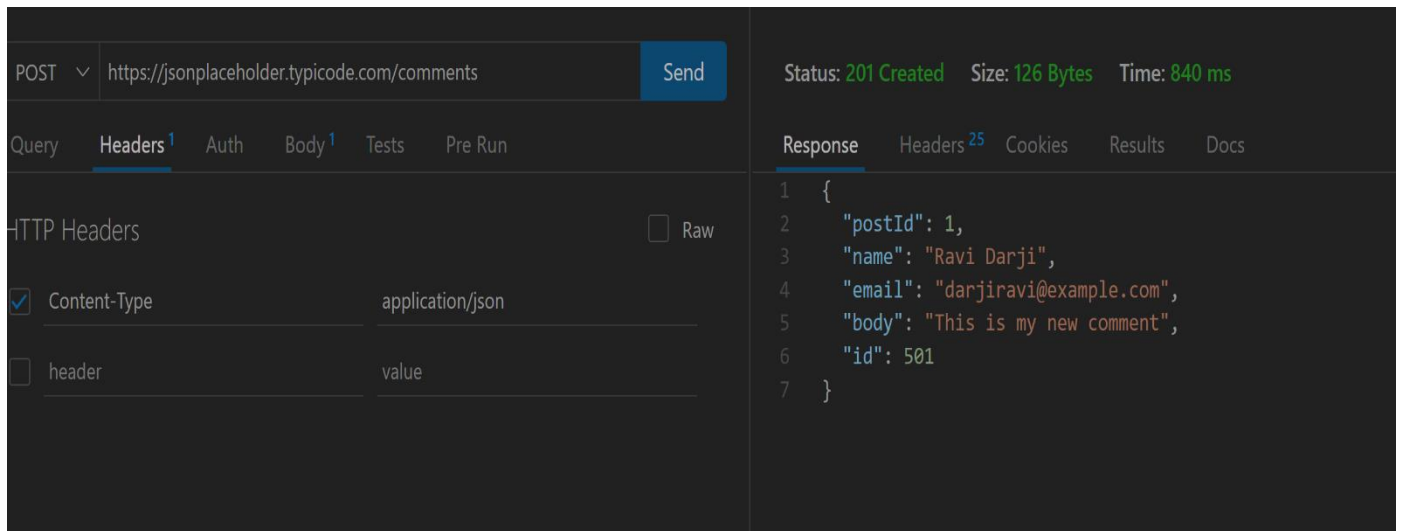
```
POST https://jsonplaceholder.typicode.com/comments
Send

Body
JSON
1 {
2   "postId": 1,
3   "name": "Ravi Darji",
4   "email": "darjiravi@example.com",
5   "body": "This is my new comment"
6 }

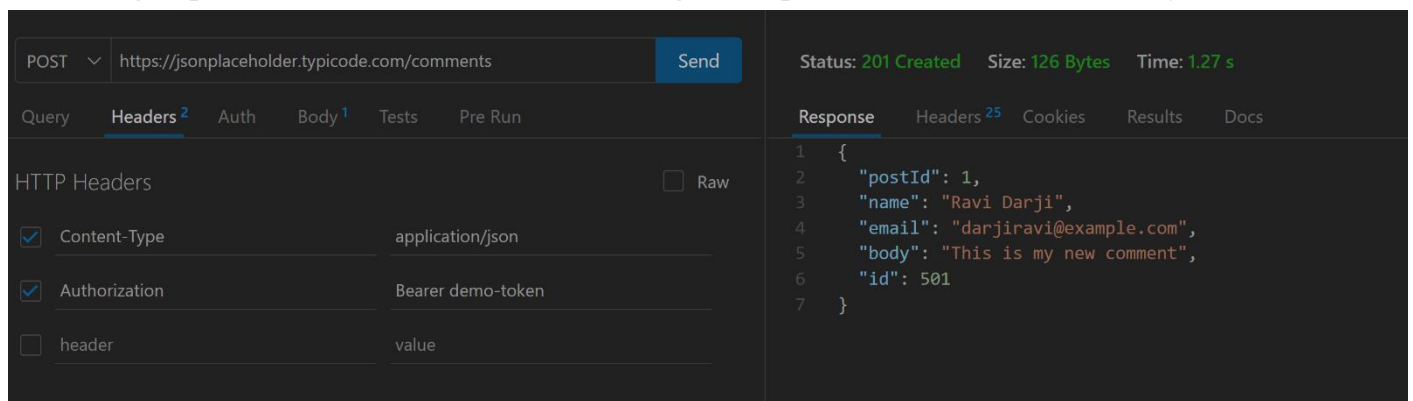
Response
1 {
2   "postId": 1,
3   "name": "Ravi Darji",
4   "email": "darjiravi@example.com",
5   "body": "This is my new comment",
6   "id": 501
7 }
```

c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.

→ Both requests returned the same response and status code because JSONPlaceholder accepts the request even without the Content-Type header.



d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.



e) Record the status code for every step above (a–d) in a short table or list.

Step	Request	Status Code
a)	GET /comments?postId=1	200 OK
b)	POST /comments (No Headers)	201 Created
c)	POST /comments (Content-Type)	201 Created
d)	POST /comments (Content-Type + Authorization)	201 Created

f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

→ In a real API, the server would verify the Authorization token before allowing the request. If the token were valid, the request would succeed; otherwise, the server would return an error such as 401 Unauthorized or 403 Forbidden, and the comment would not be created.